

ITALIAN VOICE ASSISTANT

V1 Build Guide — Production Blueprint

Sprint Length: 3–4 weeks (2 wks min)

Latency Target: < 5.0 seconds

Budget Cap: €5 OpenAI Hard Cap

Core Principle: A working, limited assistant that fails gracefully is infinitely better than an ambitious one that never launches. Changes from the baseline configuration are flagged with specialized badges.

1. PROJECT FOUNDATIONS & V1 OBJECTIVES

The core objective of V1 is to architect a functionally complete, highly responsive Italian conversational assistant optimized for learner speech engagement. The assistant focuses entirely on auditory loop interactions to build modern production skills around latency mitigation, prompt scaling, and disciplined pipeline engineering.

Functional Scope Matrix

- **Speech Intake:** Captures audio streams directly via client browser constraints **[AMENDED]**. Text input fallback mechanics are stripped out from production.
- **Linguistic Normalization:** Translates incoming webm blobs natively into optimized Italian text.
- **Contextual Orchestration:** Evaluates user grammar quality, delivers exactly one concise corrective note, and transitions instantly into a contextual follow-up prompt.
- **Acoustic Feedback:** Synthesizes clean Italian speech responses via low-latency streams.

2. ARCHITECTURAL & BUDGET CONSTRAINTS

The 5-Second Latency Envelope

Warm-session loops must establish playback in under 5 seconds. Telemetry monitoring must isolate and measure each architectural touchpoint:

Pipeline Milestone	Performance Indicator Target	Optimization Strategy
Recording Upload	Instantaneous Stream Transfer	8–10 Second Max Hard-Cap Chunk Allocation
STT Transformation	Sub-second Processing	Native WebM Whisper API Ingestion
LLM First Token	Streamed Real-Time Ingestion	Strict max_token execution bounds
TTS Serialization	Parallel Chunk Buffer Initiation	Enforce highly condensed response limits
Audio Playback	Total Loop Execution < 5.0s	Client-side runtime audio queuing

Financial Boundary Control

- **Token Constraints:** Explicit max_tokens ceiling enforced on all outbound structural queries.
- **Turn Management:** Volatility buffer restricting conversation memory limits to a 4–6 turn rolling index window.
- **Acoustic Efficiency:** Client-side audio asset caching models to eliminate redundant TTS compute expenses on historical replays.

3. PRODUCTION SOFTWARE STACK

Layer Element	Technology Selected	Production Architecture Role
Core Backend	FastAPI Framework	Asynchronous REST routing and high-performance execution.
LLM Orchestration	OpenAI GPT-4o-mini	Contextual grammar valuation and dynamic conversational branching.
Speech-to-Text (STT)	OpenAI Whisper Engine	Native parsing of unstructured user voice inputs.
Text-to-Speech (TTS)	OpenAI TTS Standard	Fluid, low-latency target language speech generation.
Frontend Core	Vanilla HTML5 / CSS3 / ES6 Javascript	Dependency-free presentation architecture avoiding virtual DOM overhead.
Audio Acquisition	Browser MediaRecorder API	Raw hardware capturing packaged inside a WebM envelope.
Deployment Architecture	Hugging Face Spaces + Docker SDK	Isolated container infrastructure with robust environment sandboxing.

4. BLUEPRINT SYSTEM ARCHITECTURE

The architecture enforces an asynchronous decoupled interaction scheme between the frontend viewport layout and the FastAPI services layer:

```
Browser Client Interface
├─ Native Background Scene Ingestion Canvas
├─ Character Viewport Node (Theresa Avatar Anchor) [AMENDED: VOICE-ONLY MODEL]
├─ Micro-Sized Target Proficiency Selector
└─ Direct Action Hardware Button (Press-to-Talk)

FastAPI Routing Engine
├─ POST /chat      → Ingests Context Data + Active Level Matrix → Generates Token Stream
├─ POST /transcribe → Ingests Audio Blobs → Executes Whisper Engine → Text Normalization
└─ POST /speak     → Ingests Text Target String → Pipelines TTS Asset Stream
```

PHASE 0: ENVIRONMENT ARCHITECTURE & INFRASTRUCTURE SETUP

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

Software Platforms: Docker Desktop, VS Code IDE, Git Core SC

Libraries & SDKs: fastapi, uvicorn[standard], openai, python-dotenv

Core Runtimes: Python 3.11 Environment

Credentials: Private OpenAI API Secret Key Matrix

Execution Protocol

- Initialize isolated project directory architecture and configure global upstream tracking to GitHub repositories (Target Profile Handle: Sai-dgt).
- Instantiate system metrics storage targets: logs/test_log.md, logs/latency_log.md, and logs/cost_log.md.
- Set up server environmental controls using local file declarations (.env mirroring .env.example paradigms).

Production Volume Exclusions (.dockerignore)

```
.env
__pycache__/*
*.pyc
temp/
tmp/
*.webm
*.mp3
*.wav
.cache/
```

PHASE 1: COGNITIVE PERSONA DESIGN & SYSTEM PROMPT ENGINEERING

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

Software Platforms: OpenAI Developer Playground Environment

System Paths: api/prompts/system_[beginner/intermediate/advanced].md

Storage Profiles: Markdown Payload Manifest Arrays

Testing Volumes: 25 Specific Behavioral Stress Datasets

Persona Blueprint Matrix: Theresa Schiavone [NEW]

Attribute Domain	Persona Specification Definition	Conversational Exploitation Vector
Identity Profile	27-Year-Old Female, Native Citizen of Turin, IT	Establishes linguistic baseline authenticity
Core Interest Nodes	Renaissance Art, Film, AS Roma Football, Piano, Chess, Culinary Arts	Rich context vectors for open-ended interaction pathways
Linguistic Stance	Warm, supportive yet structurally uncompromising educator	Enforces standard corrections through natural contextual phrasing

Structural Behavior Protocol Rules [AMENDED]

- Linguistic Integrity:** The engine must restrict outbounds strictly to verified Italian payloads.
- Adaptive Correction Loops:** Errors must be rephrased naturally within the conversational context, followed immediately by a targeted conversational prompt. Formulaic error readouts are strictly forbidden.
- Context Window Memory Tracking:** The system must suppress repeating historical questions. It must also handle reciprocal inquiries (e.g., "E tu?") gracefully without looping back into the original question.
- Volumetric Length Controls:**
 - Beginner Level:* Exactly 2 concise structural sentences.
 - Intermediate Level:* Exactly 3 structural sentences.
 - Advanced Level:* Exactly 4 structural sentences containing idioms.

PHASE 2: TEXT ORCHESTRATION BACKEND DEVELOPMENT

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

Framework Extensions: Pydantic BaseModel Validation Layer

Routing Assets: FastAPI Asynchronous Routing Nodes

Engine Modules: Asynchronous OpenAI Client Initializers

Endpoints: POST /chat Route Vector

Structural Payload Signatures

```
class ChatRequest(BaseModel):
    message: constr(max_length=500, min_length=1)
    level: Literal['beginner', 'intermediate', 'advanced']
    history: List[Dict[str, str]] # Rolling history max 4-6 turns
```

Execution Architecture Details

- Implements a clean dynamic prompt ingestion pipeline that pulls files directly from `api/prompts/` based on runtime parameter evaluations.
- Enforces strict output volume rules via explicit token allocations to prevent runtime context overflows.
- Leverages async token payload streaming to support real-time user ingestion strategies.

PHASE 2.5: INFRASTRUCTURE HOST VALIDATION SPIKE

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

Hosting Providers: Hugging Face Spaces Cluster	Automation Tooling: Latency Log Analyzers
Diagnostics Matrix: /ping or Core Skeleton Endpoints	Telemetry Files: logs/latency_log.md Updates

Deploy a skeleton container instance early to evaluate cold-start delays and real-world networking overhead. Conduct sequential request cycles to establish clean warm-session operational baselines. If these early connectivity loops breach the 5-second boundary, apply optimizations immediately before building out downstream services.

PHASE 3: SCREENLESS CHARACTER FRONTEND DESIGN

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

Graphics Frameworks: CSS Fluid Layout Architectures	Image Models: Gemini Image Models (Scene/Avatar Asset Generation)
Media Extensions: Browser Native Hardware MediaRecorder API	Asset Directory: static/assets/background.png, theresa.png

UI Interaction Specs **[AMENDED]**

The interface operates purely as an auditory loop. The layout eliminates standard UI paradigms like text chat bubbles, input text fields, and chronological transcript lists.

- **Visual Background Canvas:** Immersive Italian ambient backdrop asset (`background.png`).
- **Character Viewport Element:** A centered, layered avatar component representing Theresa (`theresa.png`).
- **Control Layout:** A single, low-profile configuration element for setting target proficiency levels, paired with an input acquisition action button (Press-and-Hold to talk).

State Machine Architecture

Runtime State	UI Visual Representation	State Operational Bounds / Constraints
idle	Static interactive anchor presentation	System open and awaiting hardware line polling signals.
recording	Dynamic glow ring pulse indicator animation	Hard temporal ceiling enforced at 8–10 seconds.
transcribing	Low-profile loading animation sequence	Asynchronous transfer optimization active.
thinking	Subtle processing state animation	Evaluating downstream structural text transformations.
speaking	Dynamic interactive waveform render	Locks hardware interface nodes; prevents conversational overlap.
error	Explicit error indicator readout code	Maps exact service failures directly to custom alert states.

Production System Error Maps

- mic_denied: Client browser interface could not secure hardware microphone lines.
- empty_audio: Input processing pipeline detected zero amplitude change inside the payload.
- transcription_timeout: Remote Speech-to-Text conversion phase breached target temporal limits.
- model_failure: Central core text intelligence layer threw anomalous exceptions or returned empty blocks.
- network_disconnect: Transport failure across local internet connections.

PHASE 4A & 4B: SPEECH INGESTION LOOP & MULTI-CHANNEL STREAMING

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

- Audio Engines: OpenAI Whisper API, OpenAI TTS Engine Matrix
- Caching Profiles: Client-Side Audio Data Blob Caching
- Backend Framework: FastAPI StreamingResponse Adapters
- Routes: POST /transcribe, POST /speak

Pipeline Execution Architecture

- POST /transcribe: Ingests inbound multi-part WebM form-data payloads. Validates raw binary sizes, flags empty buffers, and passes data to Whisper.
- POST /speak: Receives structural response payloads and yields high-fidelity audio output streams. Implements client-side binary blob mapping to eliminate redundant asset reconstruction overhead on replay.
- Phase 4B Parallel Extension (Optional Optimization Target): Breaks downstream text arrays into smaller sentence-level structures. Pipelines these segments through parallel text-to-speech workers and coordinates playback via an ordered client-side audio buffer. Alternate fallback paths are maintained to protect pipeline stability.

PHASE 5, 6 & 7: FINE-TUNING, MULTI-SCENARIO VALIDATION & DEPLOYMENT

REQUIRED TECHNICAL PROVISIONS & DEPENDENCIES

- Testing Frameworks: Boundary Exception Testing Tools
- Containers: Python 3.11 Slim Production Base Image
- Secrets Protection: HF Space Secret Environment Hooks
- Log Volumes: System Log Manifest Indexes

Validation Framework Strategy

Execute 30 distinct validation routines across all 3 language tiers (totaling 90 validation runs). Introduce structured failure scenarios to ensure robust exception handling under difficult operating conditions:

- **Zero-Signal Validation:** Simulates empty recordings to verify that the app triggers the `empty_audio` alert state without hanging.
- **Cross-Linguistic Redirection:** Injects non-Italian inputs (e.g., French/English phrases) to ensure the persona gently redirects the user back to the target language.
- **Hardware Boundary Simulation:** Forces sudden drops in backend network connectivity to confirm the UI drops cleanly into the `network_disconnect` error state.

Production Deployment Specification (Dockerfile)

```
FROM python:3.11-slim
WORKDIR /code
COPY ./requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
COPY . .
RUN mkdir -p logs && touch logs/latency_log.md logs/test_log.md logs/cost_log.md
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "7860"]
```

8. DEFINITIVE TARGET ACCEPTANCE METRICS

- **E2E Loop Integrity:** Verified voice-to-voice interaction cycle without relying on visual text fallback layers.
- **Linguistic Accuracy:** Successful application of conversational filters across Beginner, Intermediate, and Advanced tiers.
- **Prompt Rule Enforcement:** Combined evaluation cycles confirming a strict *[Correction + Dynamic Follow-up Prompt]* loop layout.
- **Security Standard Compliance:** Container configurations validated to ensure zero deployment-layer exposures of private API key materials.